ADVANCED ROBOTIC PERCEPTION / IMAGE PROCESSING AND COMPUTER VISION

TRACKING

RIKKE GADE 2026

AALBORG UNIVERSITY
DENMARK

Agenda

- Tracking – introduction
- Prediction
- Kalman filter
- Mean shift
- Multi-target tracking

Tracking

- What is visual tracking?
  - The ability to visually follow the same object over a sequence of frames
- Different branches of visual tracking:
  - Single object tracking (SOT) vs multiple object tracking (MOT)

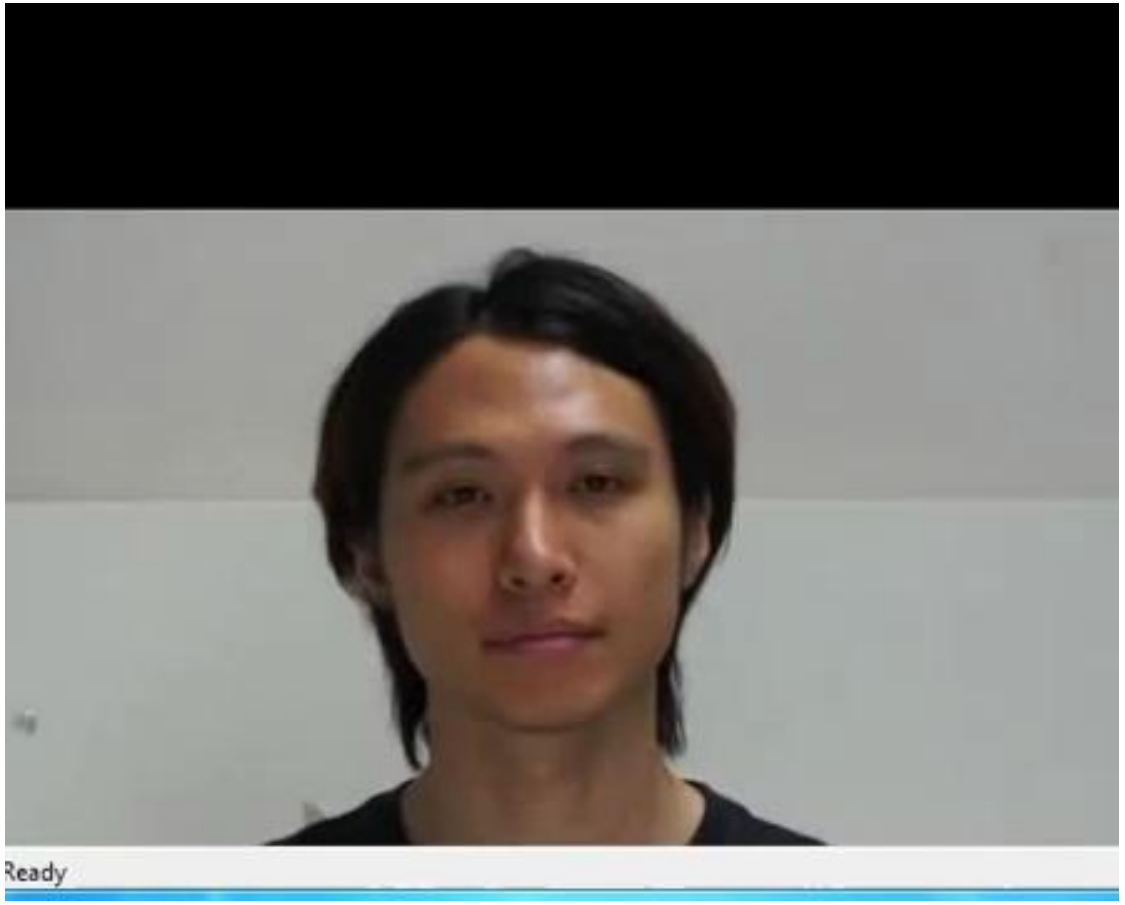
Tracking example



Tracking example



Tracking example



Tracking example



AALBORG UNIVERSITY
DENMARK

Simple tracking

- Detect the object in each frame
- Simply connect the state vector in sequential frames

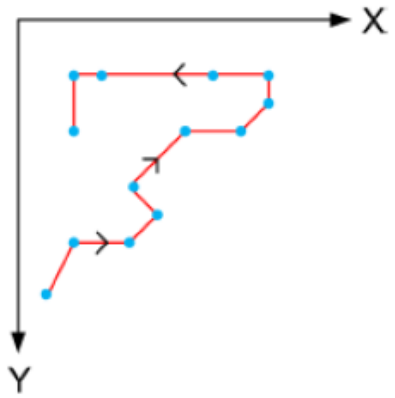
- Add information about the object in a state vector
  - Position
  - Velocity
  - Acceleration
  - Size
  - Shape
  - Colour
  - Etc.

Simple tracking

- Aim: Generate one trajectory for each observed object

Time	1	2	3	4	5	6	7	8	9	10	11	12	13
X	1	2	4	5	4	6	8	9	9	7	3	2	2
Y	10	8	8	7	6	4	4	3	2	2	2	2	4

The position of an object over time



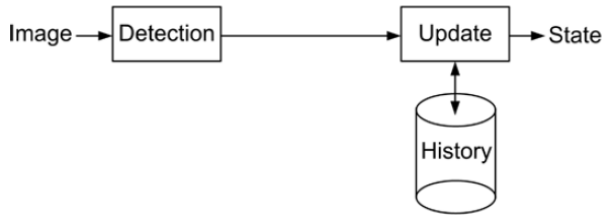
The trajectory of an object over time

Simple tracking

- Simply connecting detections is not really tracking – just a series of detections
  - Each detection is independent
  - No temporal information used

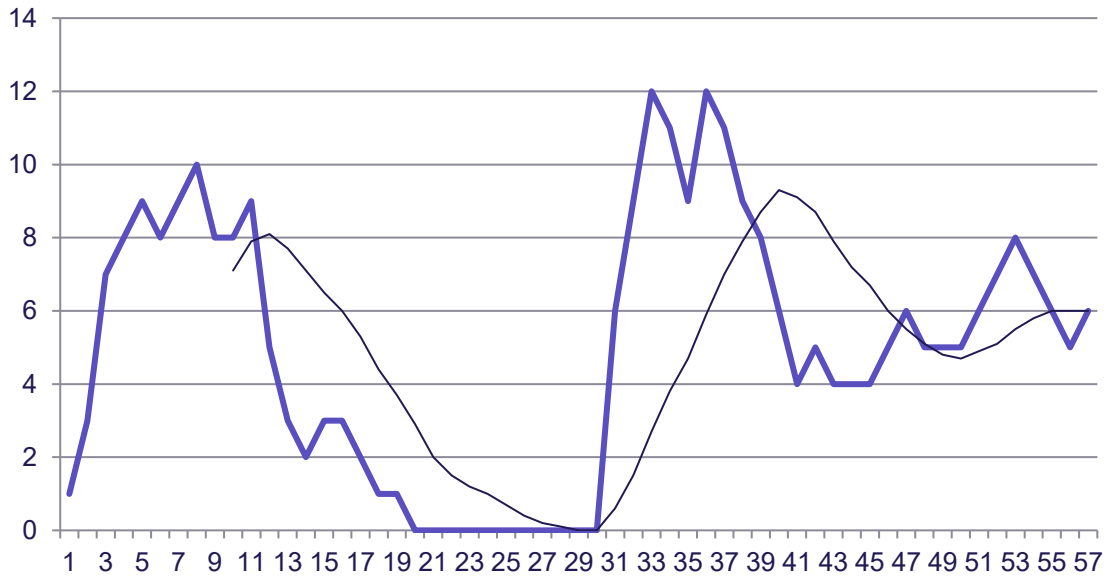
- Simplest solution:
  - Use previous states when estimating the new position
  - Smoothing – outliers and noise will be reduced

- Implementations:
  - Average of last N states (slow changes)
  - Average of past AND future states (delay)
  - Weighted N states by age (still slow)



Simple tracking

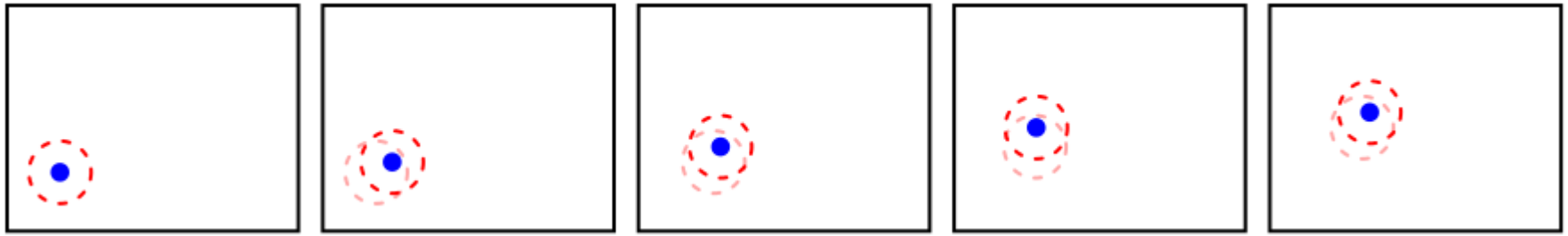
- Problem of smoothing:

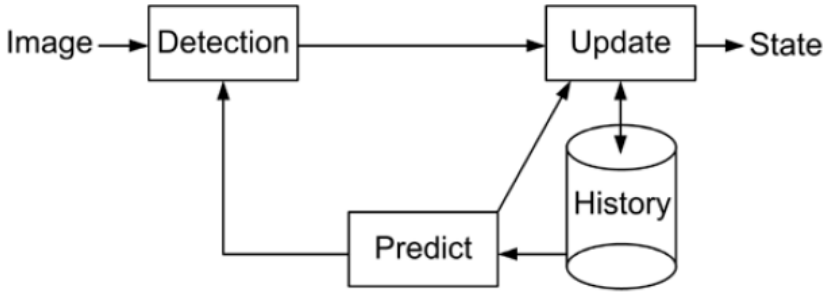


Prediction

- The object motion is usually much slower than the framerate of the camera
- Assume that the object is not moving very much between two frames

- Predict-match-update framework:
- Use the previous observations to construct a region of interest (ROI)
- If same position is assumed (0th order linear motion):





AALBORG UNIVERSITY
DENMARK

Predict-match-update

- 0th order linear motion
  - Assume no motion – same position

- 1st order linear motion
  - Include current velocity – assume constant velocity
  - Current position: $\vec{p}(t) = [x(t), y(t)]$
  - Velocity: $\vec{v}(t) = \left[v_x(t), v_y(t)\right]$
  - Predicted position: $\vec{p}(t+1) = \vec{v}(t) \cdot \Delta t + \vec{p}(t)$

- 2nd order linear motion
  - Include current acceleration – assume constant acceleration
  - Current acceleration: $\vec{a}(t) = [a_x(t), a_y(t)]$
  - Predicted position: $\vec{p}(t+1) = \frac{1}{2}\vec{a}(t) \cdot \Delta t^2 + \vec{v}(t) \cdot \Delta t + \vec{p}(t)$

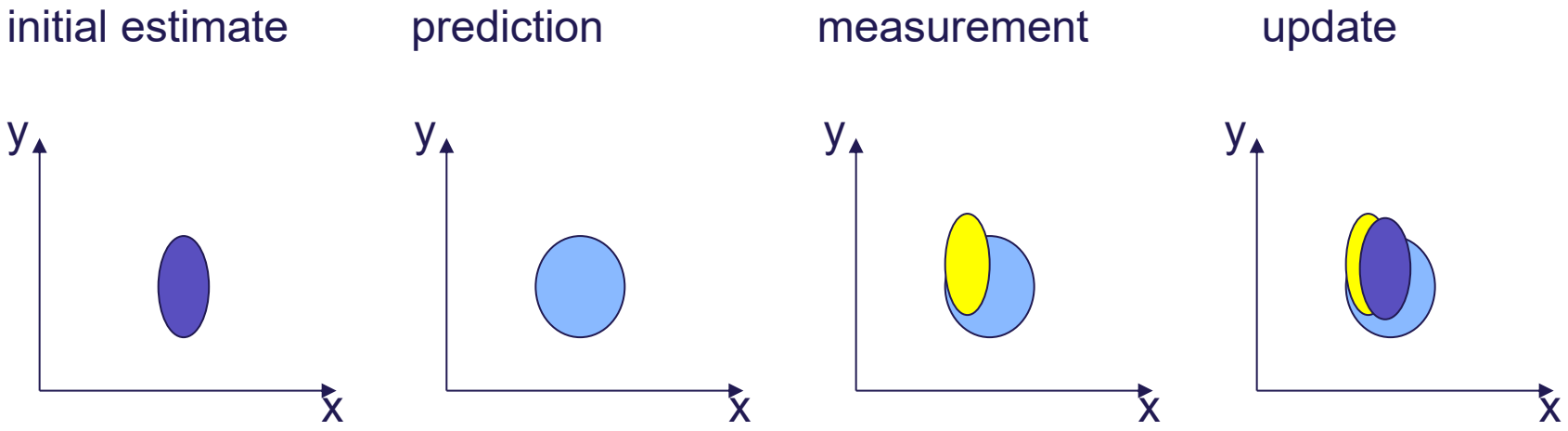
Predict-match-update

- But linear models assume predictable, stable motion.
- What about drunk people, sports games, fish, …. ??
- Non-linear models exist – or simply assume nothing!

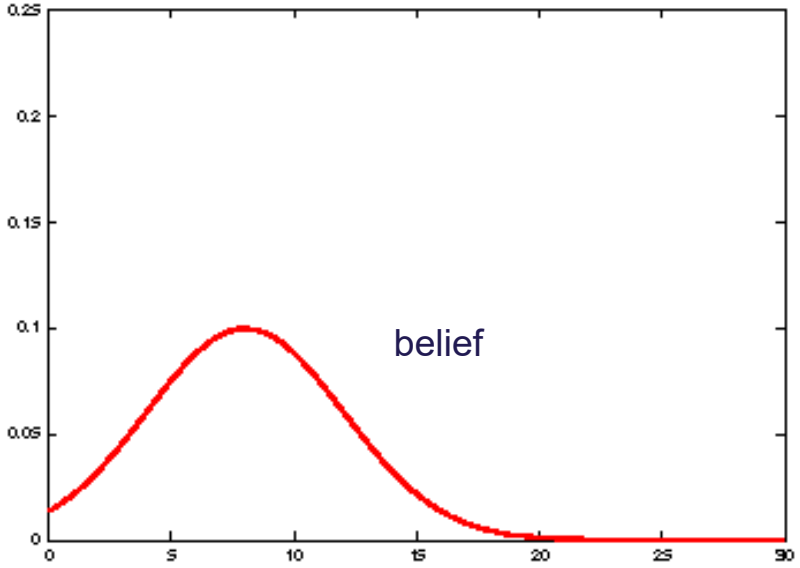
- What about the size of ROI?
- Proportional to the uncertainty
- Example of controlling the width of the ROI:
  - $width(t+1) = \alpha \cdot \left(\underline{\vec{p}}(t) - \vec{p}(t)\right) + (1-\alpha) \cdot width(t)$
- Likewise, update the state:
  - $\vec{s}(t) = w \cdot \underline{\vec{p}}(t) + (1-w) \cdot \vec{p}(t)$
- Where w is associated with the innovation (difference between predicted and observed position)

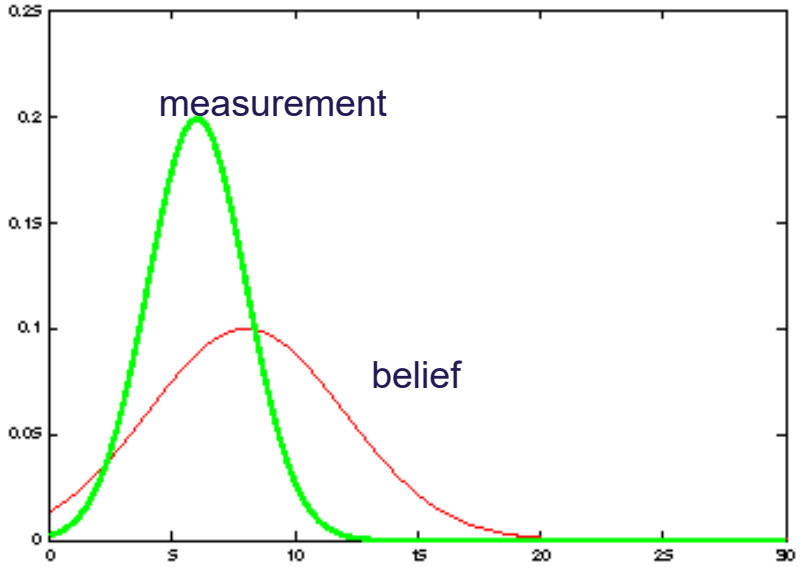
Kalman filter

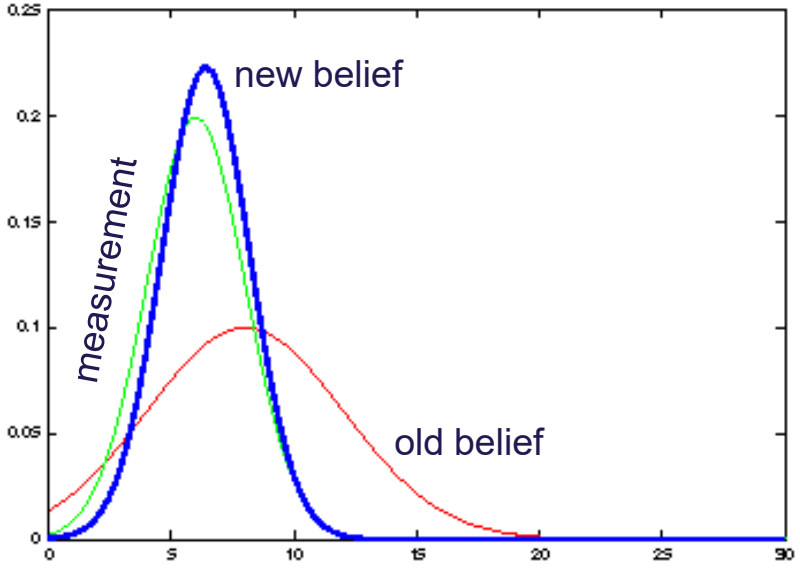
- Original paper published in 1960 by R.E. Kalman
- The algorithm is used in many different applications, e.g. navigation, time series analysis (economics), radar tracking, weather forecasting, etc.



Kalman filter updates







ORG UNIVERSITY
DENMARK

Kalman filter algorithm

- Estimate the state x of discrete-time controlled process, described by this stochastic difference equation:
  - $x_k = Ax_{k-1} + Bu_{k-1} + w_{k-1}$
- Measurement z:
  - $z_k = Hx_k + v_k$
- w_k and v_k are random variables, representing process and measurement noise respectively
  - Assumed to be independent, white and with normal probability distributions
- A is the state transition model which is applied to the previous state x_{k-1}
- B is the control-input model which is applied to the control vector u_k
- H is the observation model which maps the true state space into the observed space

Kalman filter algorithm

- Prediction
  1. Project the state ahead: $\hat{x}_k^- = A\hat{x}_{k-1} + Bu_{k-1}$
  2. Project the error covariance: $P_k^- = AP_{k-1}A^T + Q$
- Measurement Update (Correct)
  1. Compute the Kalman gain: $K_k = P_k^- H^T (HP_k^- H^T + R)^{-1}$
  2. Update estimate with measurement z_k: $\hat{x}_k = \hat{x}_k^- + K_k(z_k - H\hat{x}_k^-)$
  3. Update the error covariance: $P_k = (I - K_k H)P_k^-$

Kalman filter algorithm

- Clever filtering of noise and outliers
- Can work based on position (point) only – no appearance information needed
- If few detections are missing we can continue by relying on the prediction only and “catch up” when a new detection arrives a few frames later
- Further reading: http://www.cs.unc.edu/~welch/kalman/

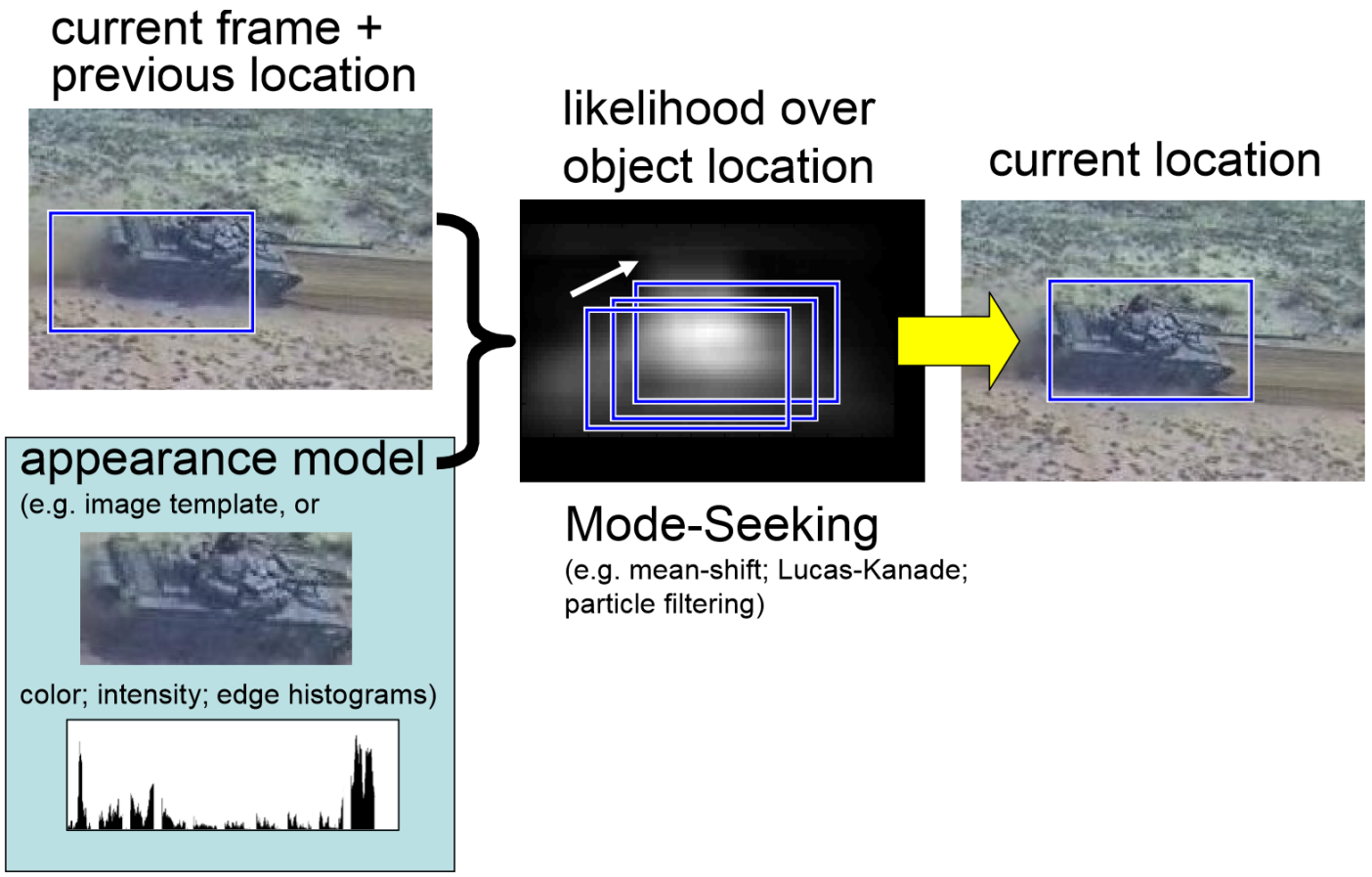
Kalman filter summary

- Estimates state of a system
  - Position
  - Velocity
  - Possibly other continuous state variables
- KF maintains
  - Mean vector for the state
  - Covariance matrix of state uncertainty
- Implements
  - Time update (prediction)
  - Measurement update

- For non-linear motion – look into the extended Kalman filter



Appearance based tracking



Mean shift

Motivation: To track non-rigid objects, (like a walking person), it is hard to specify an explicit 2D parametric motion model

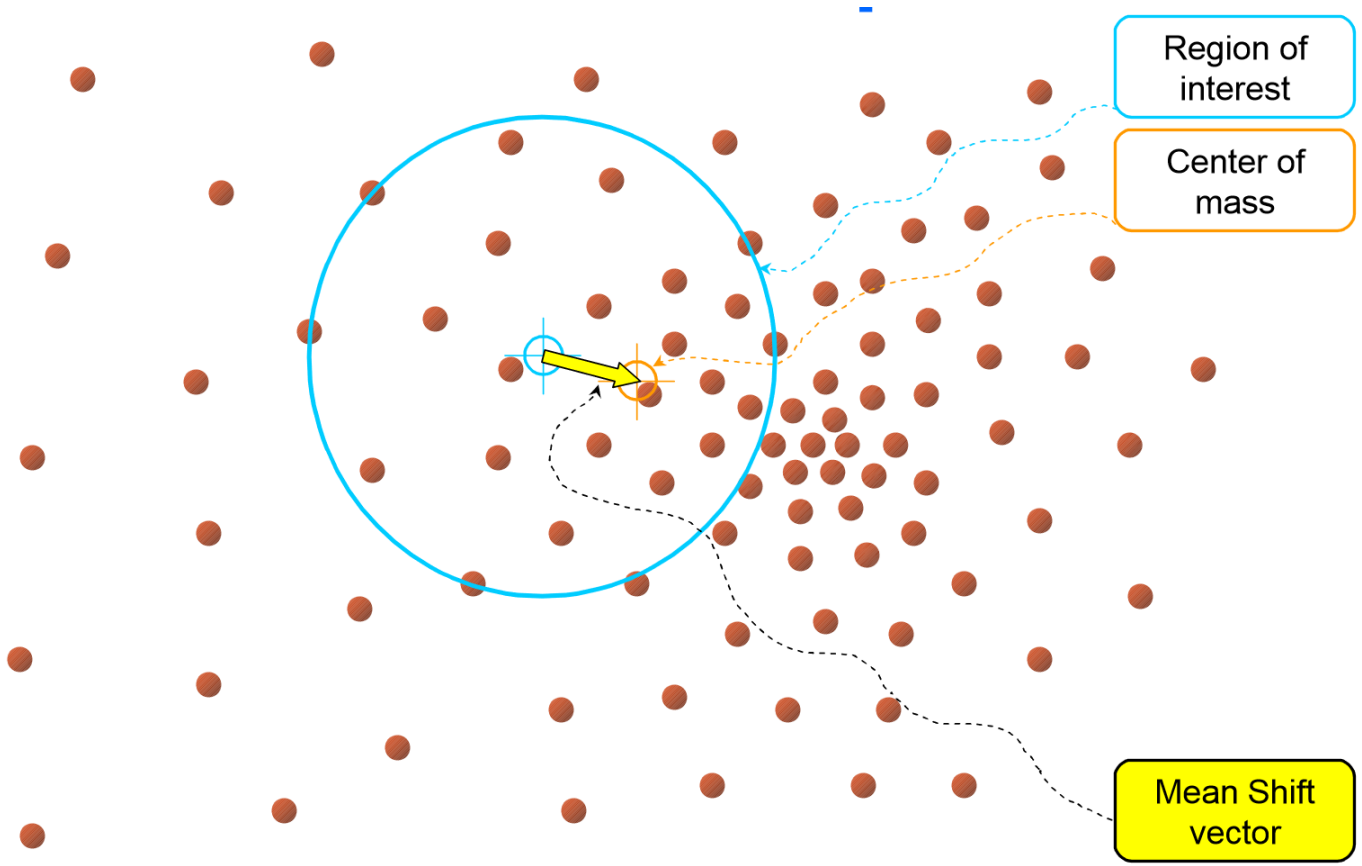
Mean shift algorithm:

- For tracking objects whose appearance is defined by histograms
  - Such as colour histograms, gradient histograms or any other histogram

- Basic idea is to shift the center towards the mean of the data

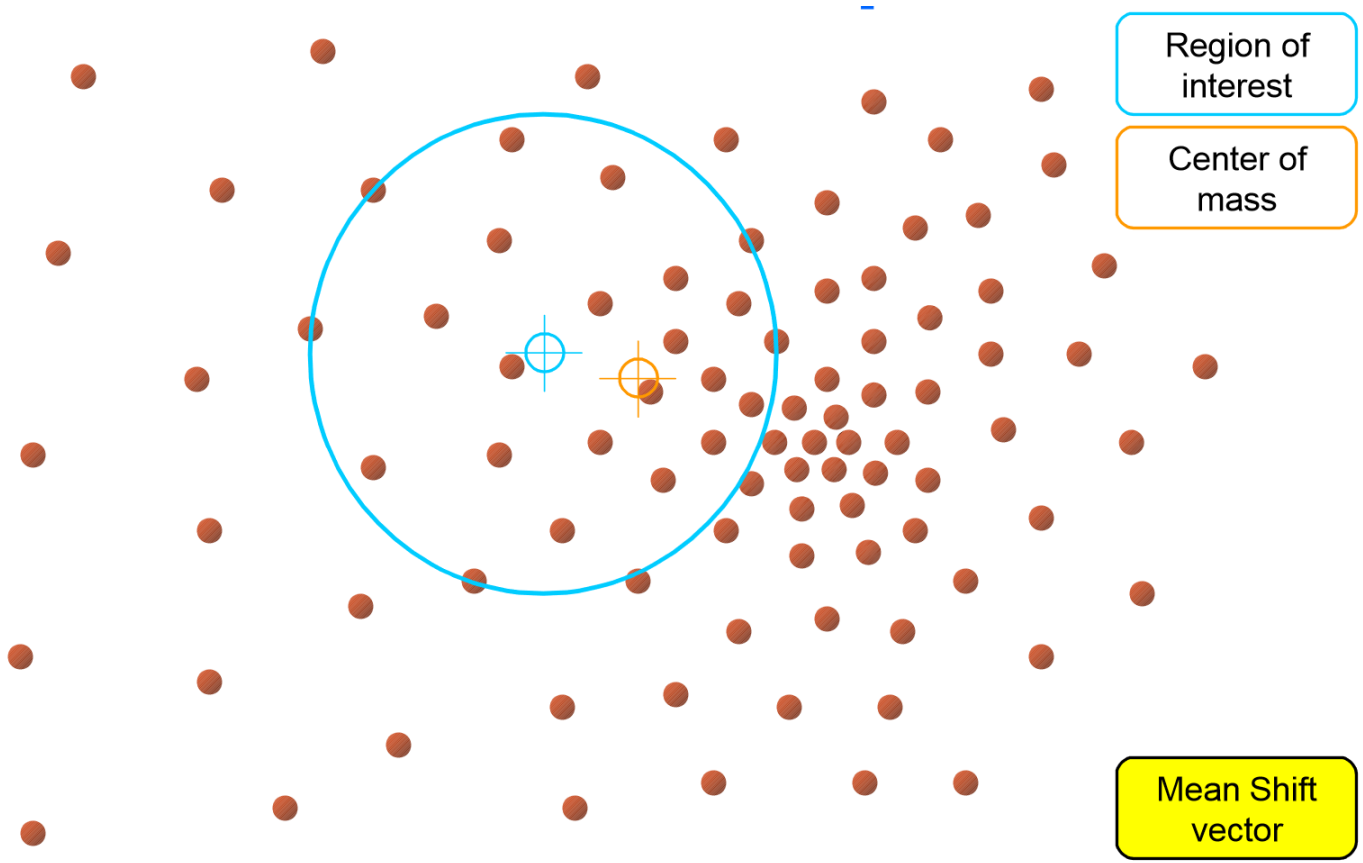


Mean shift – Intuitive description



Objective : Find the densest region

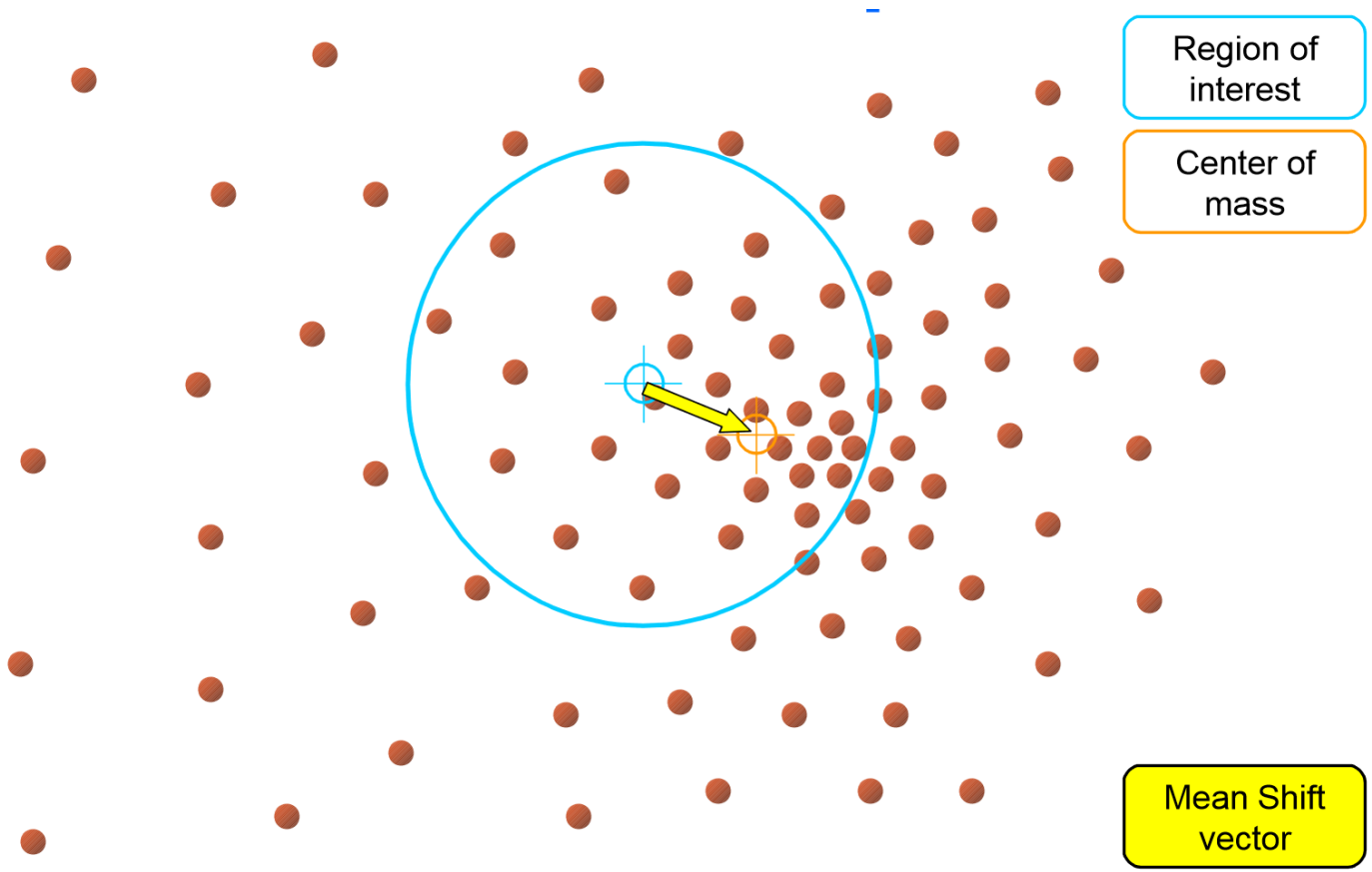
Mean shift – Intuitive description



Objective : Find the densest region

AALBORG UNIVERSITY
DENMARK

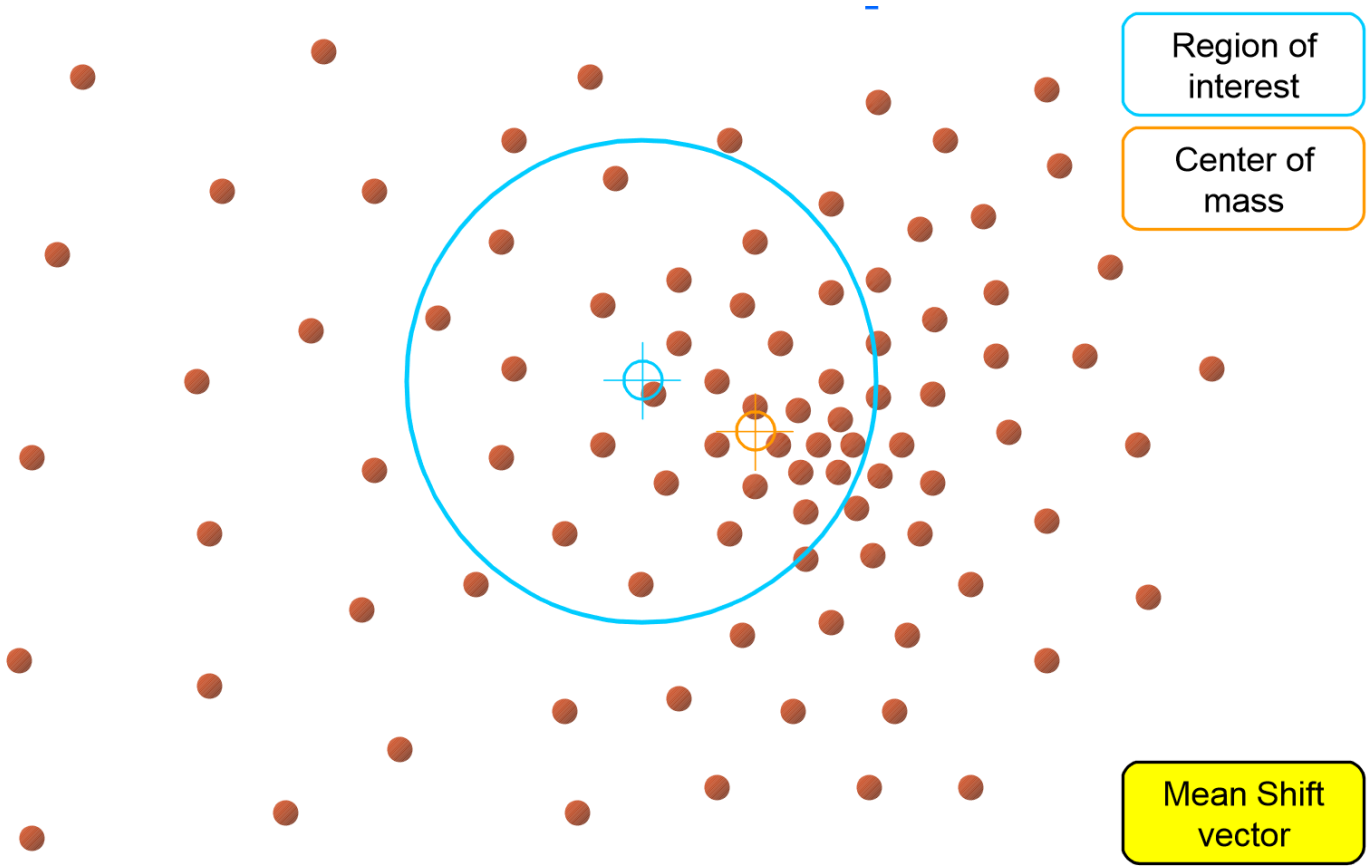
Mean shift – Intuitive description



Objective : Find the densest region

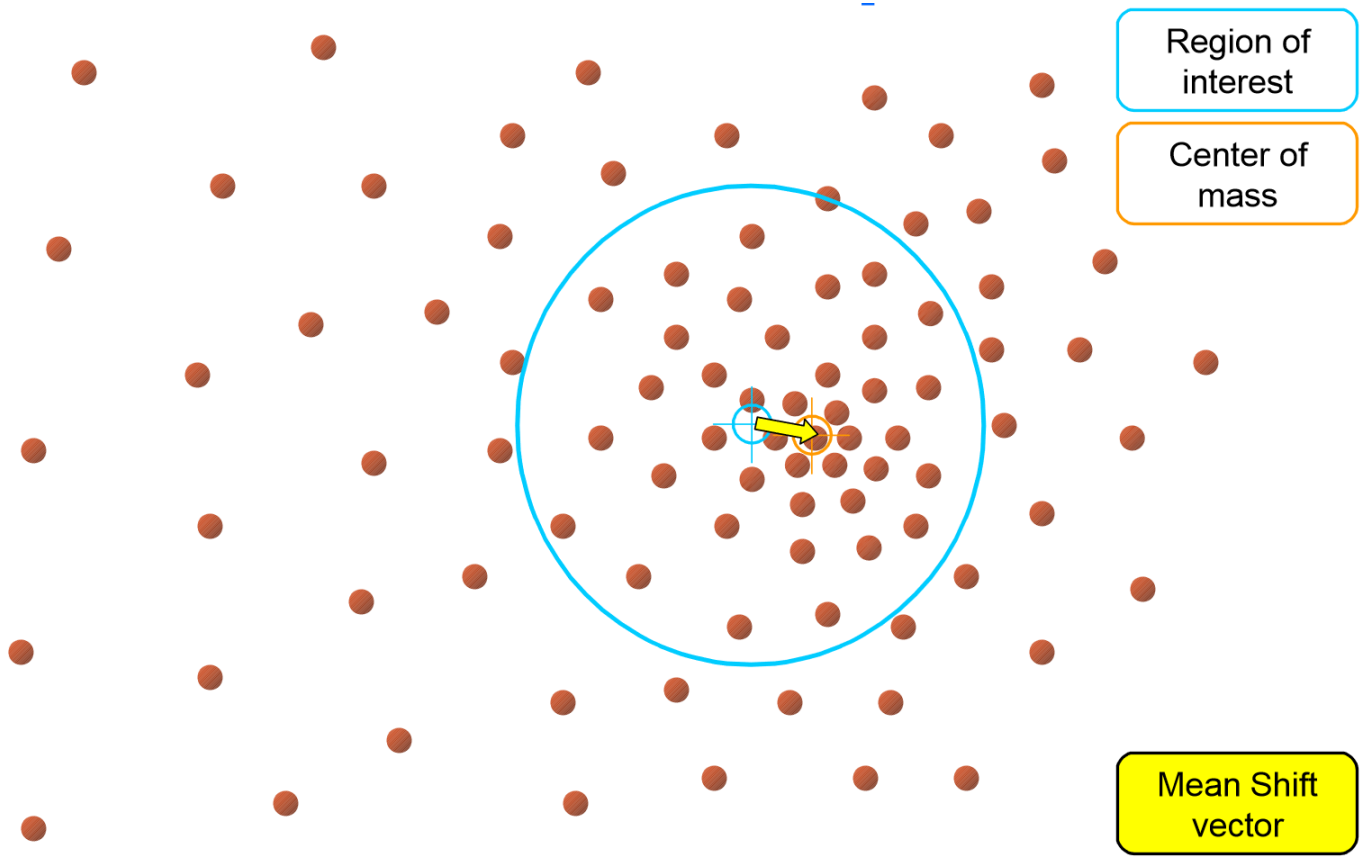
AALBORG UNIVERSITY
DENMARK

Mean shift – Intuitive description



Objective : Find the densest region

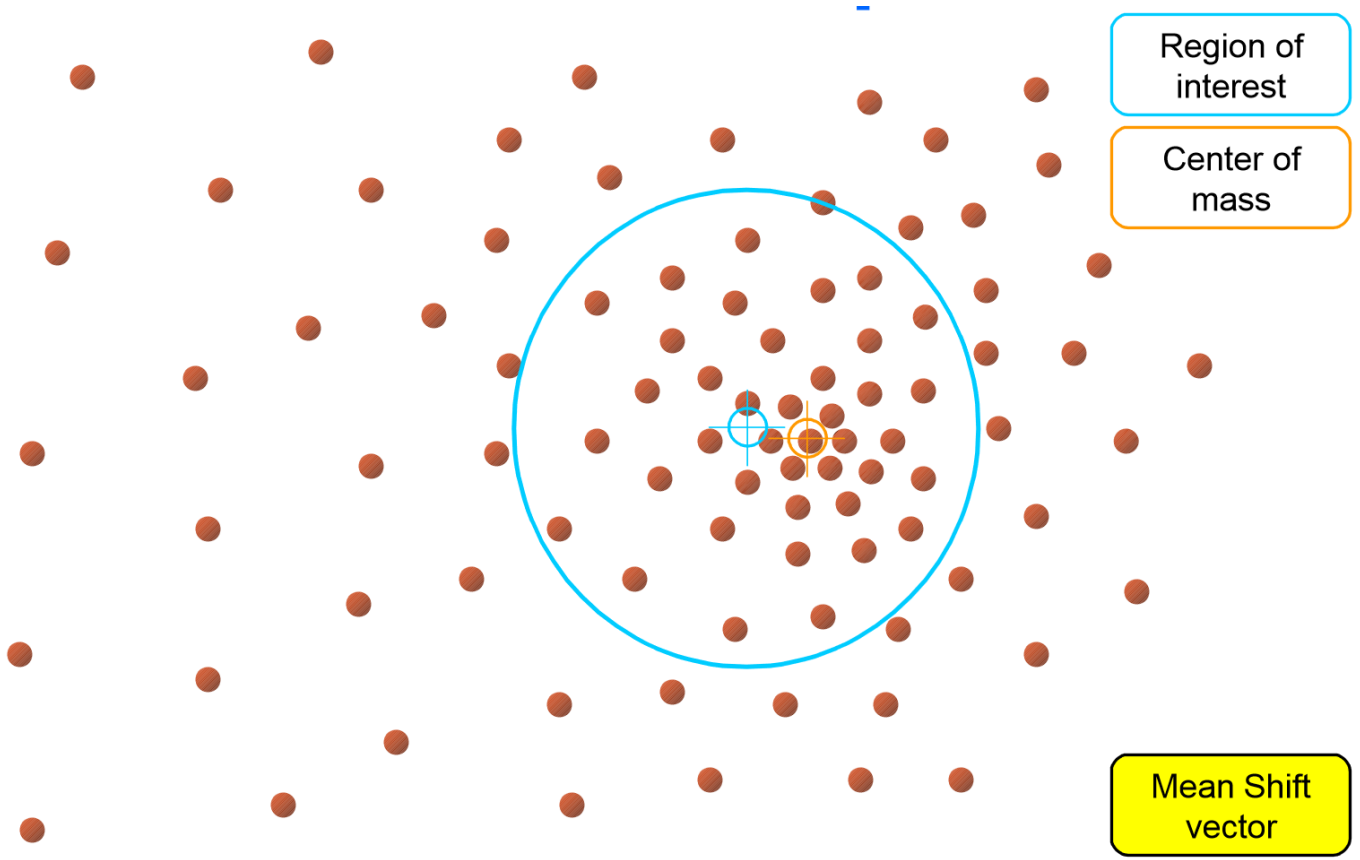
Mean shift – Intuitive description



Objective : Find the densest region

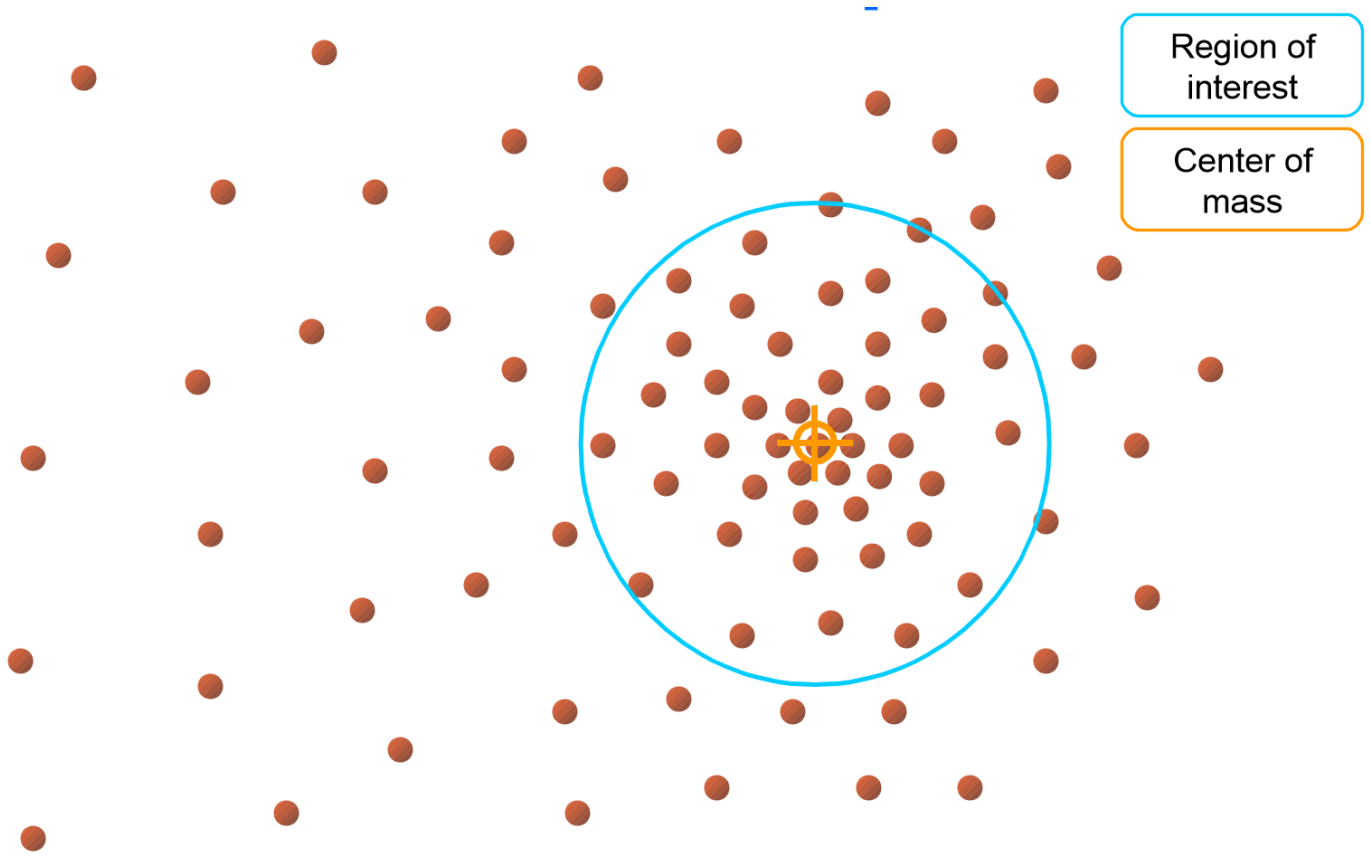
AALBORG UNIVERSITY
DENMARK

Mean shift – Intuitive description



Objective : Find the densest region

Mean shift – Intuitive description



Objective : Find the densest region

AALBORG UNIVERSITY
DENMARK

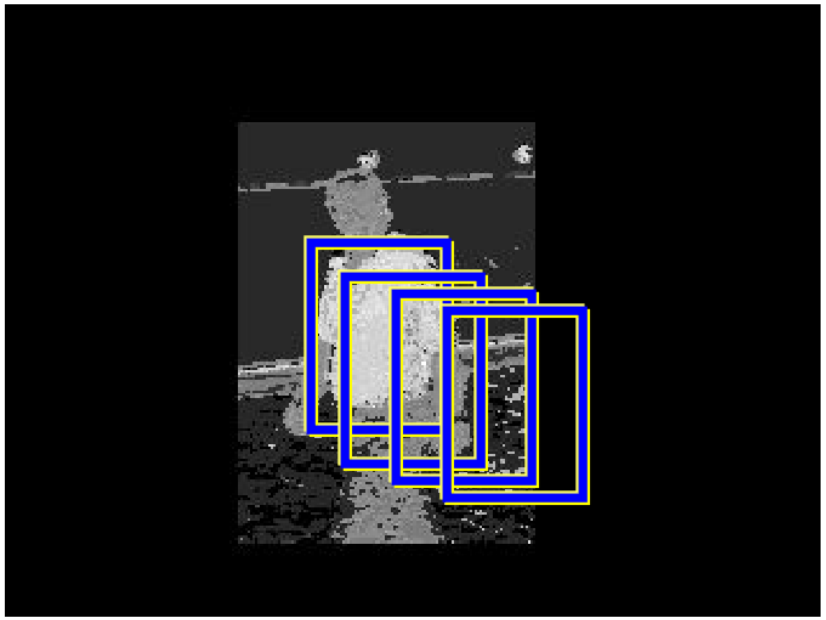
Mean shift tracking

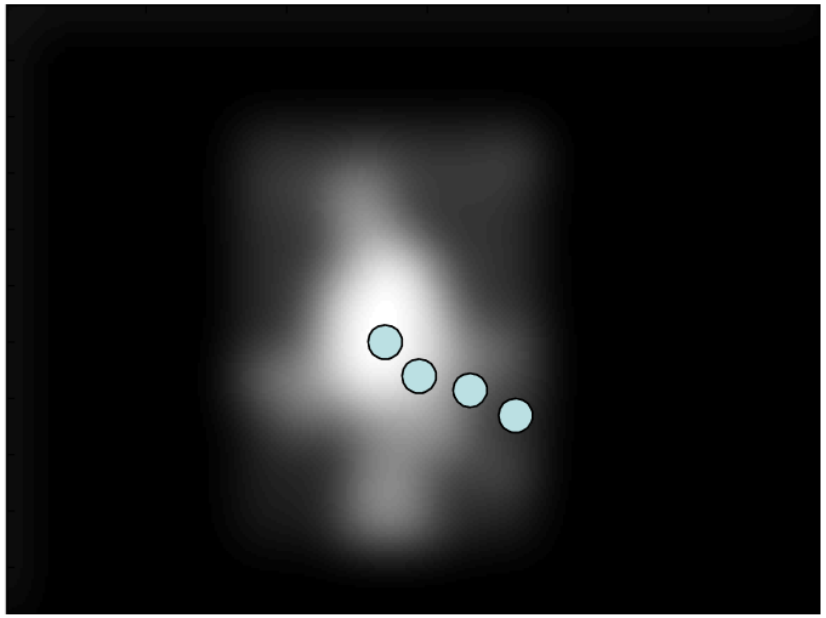
Two approaches:

1. Create a colour “likelihood” image with pixels weighted by similarity to the desired colour (best for single coloured objects).

2. Represent colour distribution with a histogram. Use Mean Shift to find region with most similar distribution of colours.

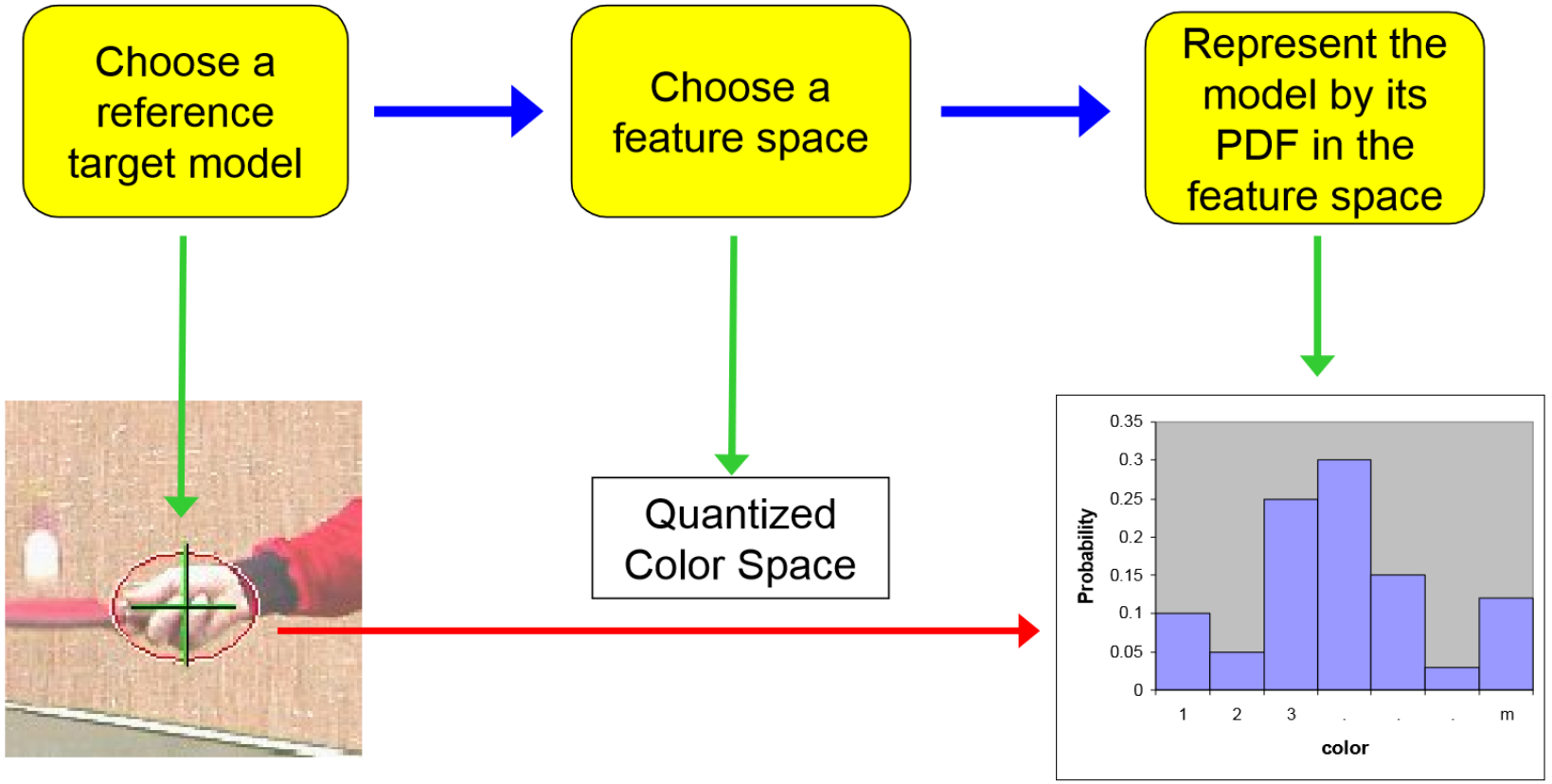
Mean shift tracking





Note: Mode we are looking for is mode of location (x,y) likelihood, not mode of the color distribution!

Mean shift tracking

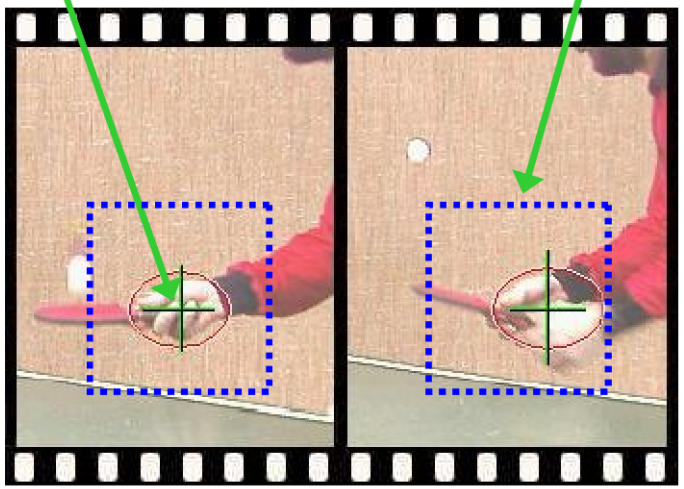


Mean shift tracking

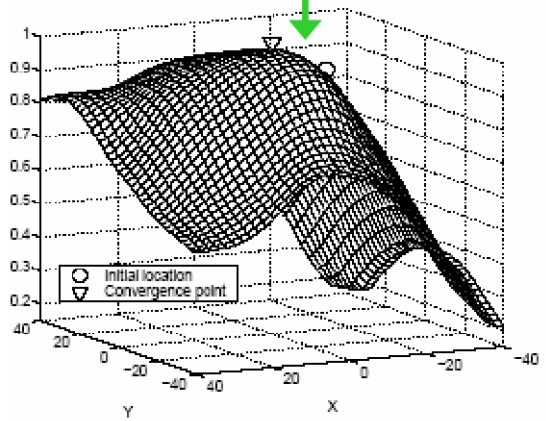
Start from the position of the model in the current frame

Search in the model's neighborhood in next frame

Find best candidate by maximizing a similarity func.



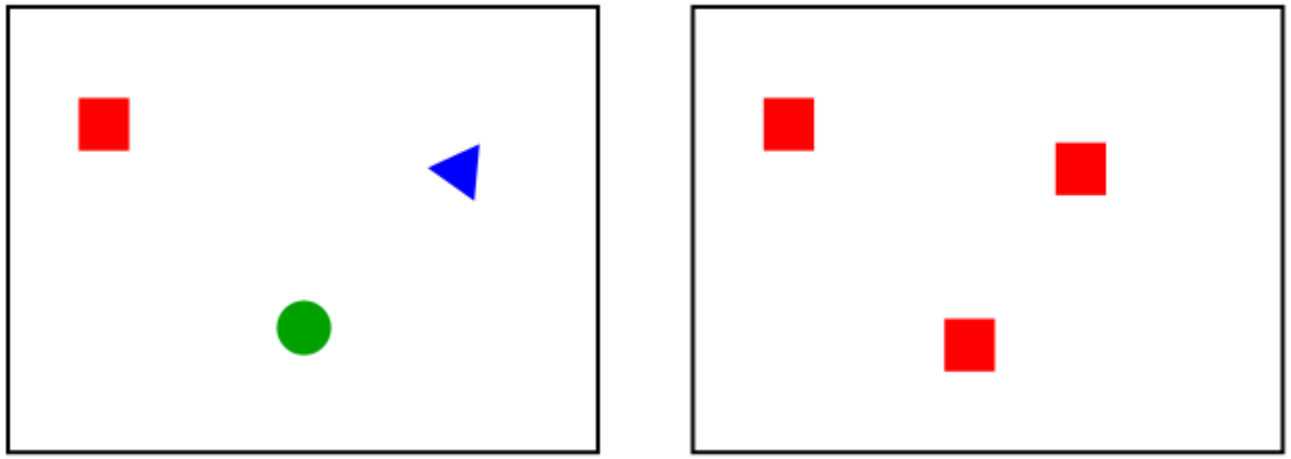
$\vec{q}$ $\vec{p}(y)$



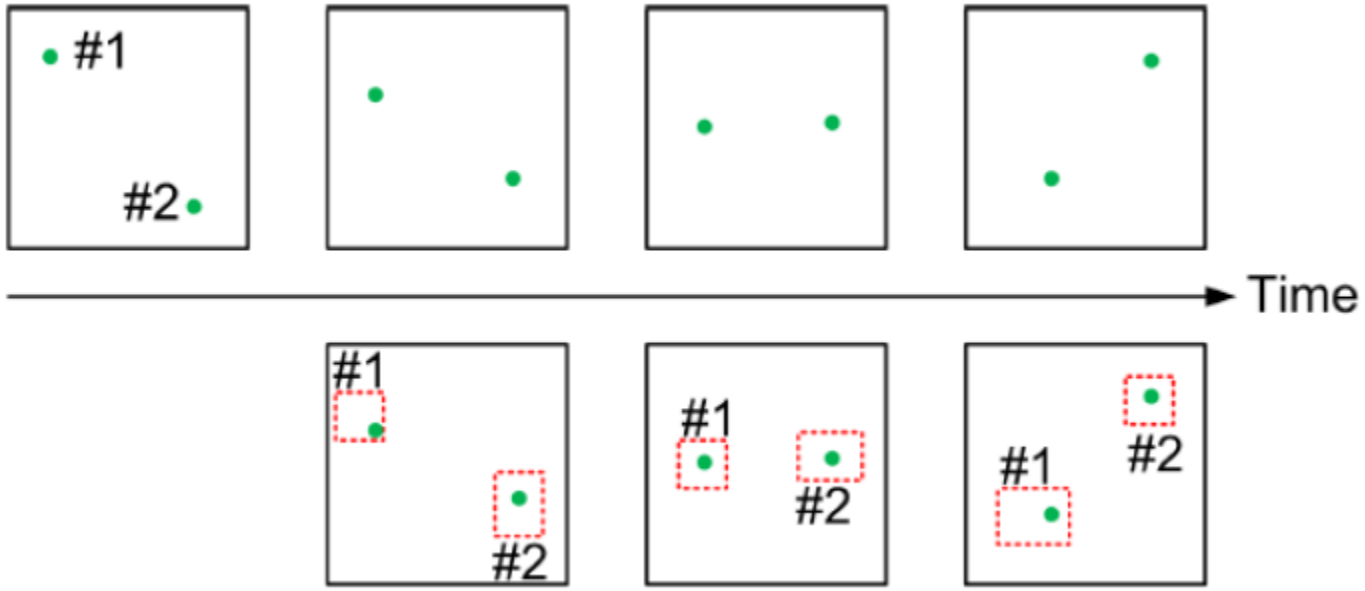
$f\left[\vec{p}(y),\vec{q}\right]$

Tracking multiple objects

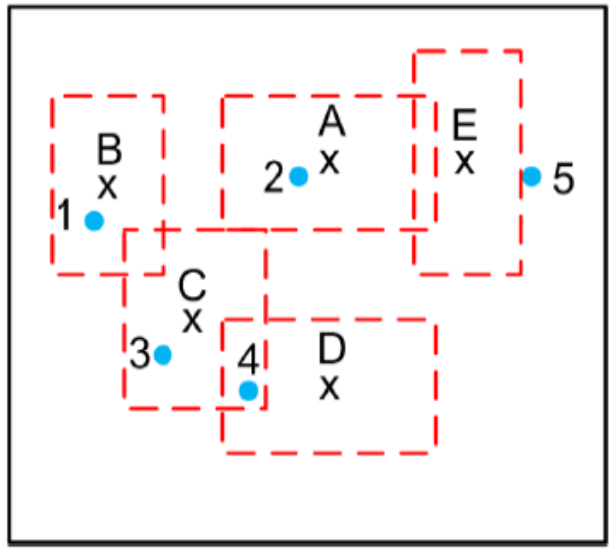
- For different objects we can just run several independent trackers
- What if objects are similar or identical?



Tracking multiple objects



Tracking multiple objects

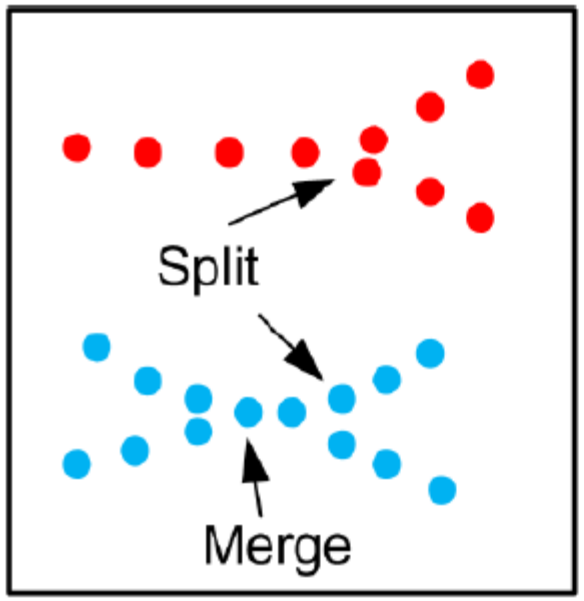


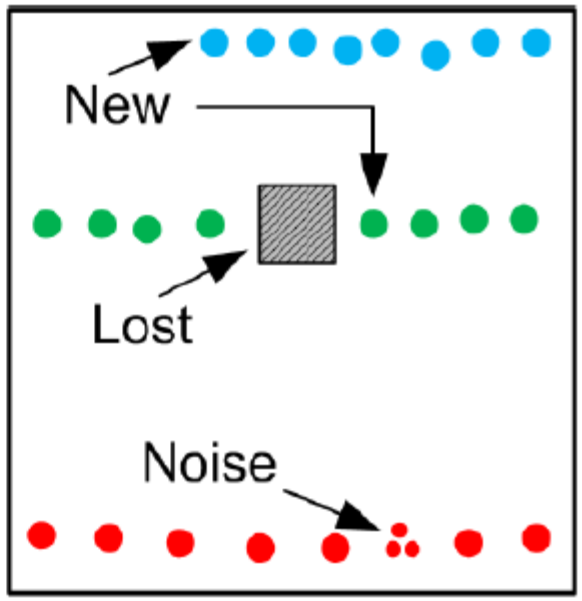
Predicted

Detected	A	B	C	D	E	
1	0	1	0	0	0	1
2	1	0	0	0	0	1
3	0	0	1	0	0	1
4	0	0	1	1	0	2
5	0	0	0	0	0	0
	1	1	2	1	0	

1 => B
2 => A
3 => C
4 => D
5: New
E: Lost

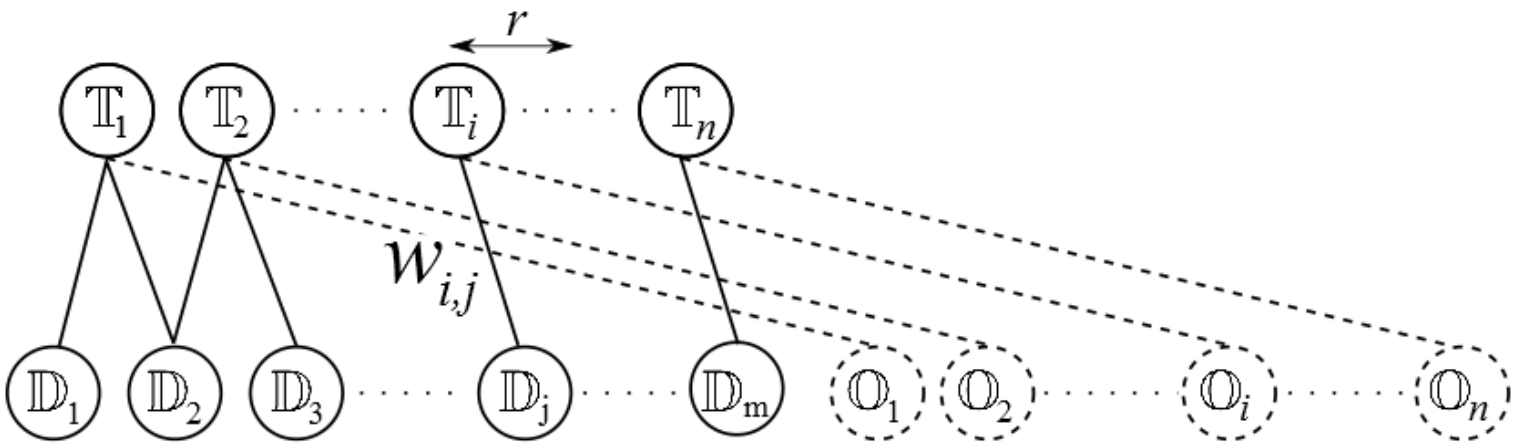
Tracking multiple objects





Tracking multiple objects

- Data association: Find best combination of trackers and detections
- Graph optimisation problem
  - One to one assignment: Hungarian algorithm
- Weights can be based on a combination of distance, appearance, etc.



Tracking multiple objects

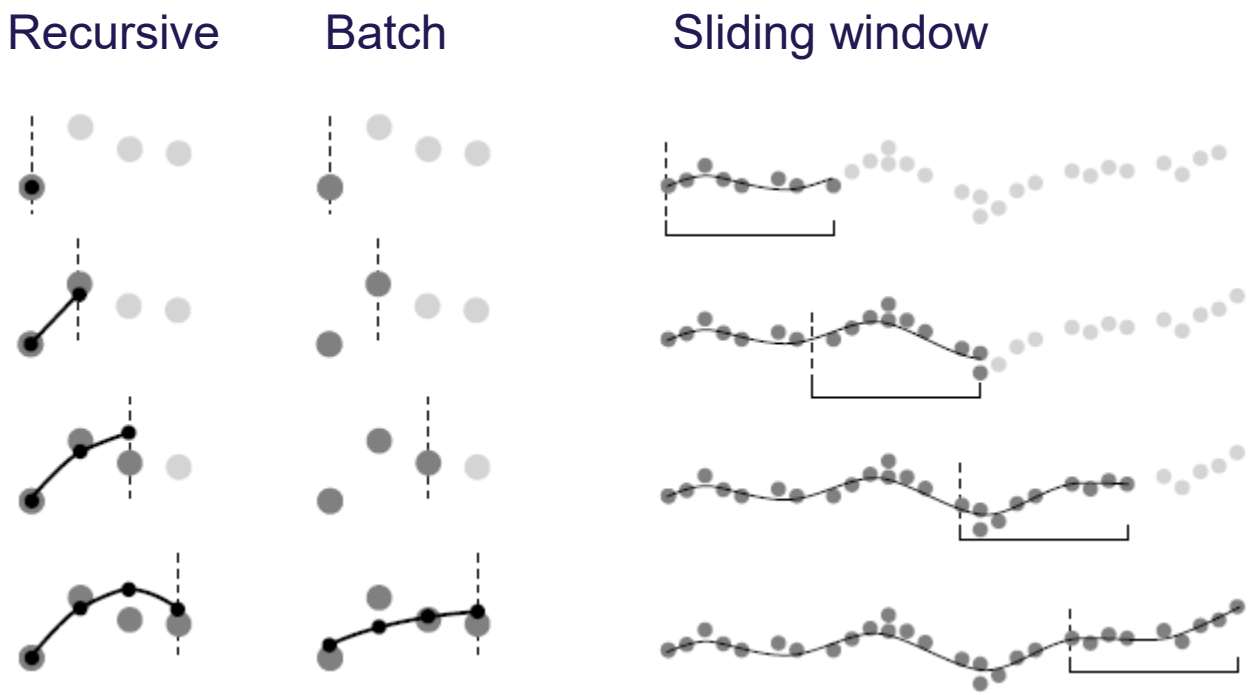
- Hungarian algorithm
  - An algorithm which finds an optimal assignment for a given cost matrix
  - Example: Weights between three trackers and three detections

	T1	T2	T3
D1	250	400	350
D2	400	600	350
D3	200	400	250

- Cost matrix: $\begin{bmatrix} 250 & 400 & 350 \\ 400 & 600 & 350 \\ 200 & 400 & 250 \end{bmatrix}$

- Detailed presentation of the algorithm: http://www.math.harvard.edu/archive/20_spring_05/handouts/assignment_overheads.pdf

Video tracking approaches



Neural network based tracking

Neural networks can be implemented to solve each task in the pipeline:

- Detection step
- Feature extraction/motion estimation
- Affinity computation
- Association/tracking step

Deep learning in video multi-object tracking: A survey
https://doi.org/10.1016/j.neucom.2019.11.023

Neural network based tracking

Common approach: Rely on robust detection algorithms

- Tracking-by-detection paradigm: Detect all relevant objects, then solve assignment problem

- Examples of popular methods:
- DeepSORT
- ByteTrack
- BoT-SORT
- Integrated in frameworks like YOLO

A re-id module is also useful to resolve full occlusions



Tracking summary

- Summary:

- Single vs. multi-target tracking
- Motion model vs. appearance

- Online
  - Recursive – necessary in real-time applications
  - Representation, search, model update

- Offline
  - Exploits information from “future” detections (batch or sliding window)
  - Can go back and forth in time – connect broken trajectories

- Multi-target tracking
  - Detections, then assignment problem